Responses to Viva Voce Questions

1 Questions from Reviewer One

1. How does the answer set programming approach compare to the bounded model checking approach for normal Petri nets? Could this be a way to unify the different approaches of the thesis?

   Answer: As mentioned in the thesis, Answer Set Programming gave an elegant and compact encoding of the PN semantics where the transition rules can be written directly. We are able to verify safety properties using this simplistic encoding. When compared to the SMT encoding, the transitions of the model and how to unfold them, as well as the encoding of the formulas is cumbersome. In terms of complexity of the code, the ASP approach for PNs contained 40 Lines of Code (LoC), whereas, the BMC approach written in CPP in DCModelChecker1.0 and DCModelChecker2.0 were 8688 Loc and 9451 LoC respectively. Using the SMT approach, we are able to verify properties beyond safety.

2. Instead of using SMT over LIA, could we use Pseudo Boolean solvers or (M)ILP solvers for the various problems addressed in the thesis?

   Answer: For the FireabilityCardinality properties as well as LIA properties with invariants, it would make sense to use Pseudo Boolean solvers or Interger Linear Programming Solvers, since they are expected to perform better in case of counting and optimization queries. Most of the queries we fed to DCModelChecker 1.0 and DCModelChecker2.0 were fireability queries over LTL, since they were lifted from the contest. We do not expect an advantage there.

   During the PhD, I have worked on an optimization problem using Gurobi, CPlex with a startup based out of TUM.

3. From a modeling point of view are there are other case-studies other than the autonomous parking system for which the ideas in this thesis can be applied?

   Answer: We have added a travel agency case study, where there are conflicts, synchronization in the model. Experiments on this model are added to our latest arxiv draft.

2 Questions from Reviewer Two

1. Why is the work on DCModelchecker1.0 and DCModelchecker2.0 not published? What could you do to make it publishable?

   Answer: One of the difficulties is that DCModelChecker1.0 and DCModelChecker2.0 are not as fast as the other state of the art tools. Our tools print the counterexample traces also, which takes significant time. We have improved the tool confidence and we have submitted the papers (along with the tools) to journals and await their response.

2. Did you participate in the Model Checking competition? What was your standing?

   Answer: We did not participate in the Model Checking contest. This is primarily since the contest benchmarks are all bounded nets, whereas, our tools seem to perform relatively better in unbounded nets. If we are able to fine tune our encoding for bounded nets, we can submit it to the contest.